kurvst Typst API

`kurvst` is a Typst package backed by `kurvst.wasm`, a small Kurbo-based plugin. Its public API is path-based: construct a path dictionary, pass that path into transforms, and draw the returned path.

It exposes Bezier utilities that are awkward or unavailable in native Typst:

- constructing cubic and Hobby paths,
- trimming paths by arc length,
- generating sampled path patterns,
- fitting Kurbo offset/parallel paths,
- converting returned path geometry to CeTZ drawing commands.

All points are dictionaries with numeric `x` and `y` fields:

```
#let p = (x: 1.2, y: -0.4)
```

Cubic segments use `start`, `ctrl-a`, `ctrl-b`, and `end`:

```
#let segment = (
  start: (x: 0, y: 0),
  ctrl-a: (x: 1, y: 0.5),
  ctrl-b: (x: 2, y: -0.5),
  end: (x: 3, y: 0),
)
```

Use `cubic-path(..segment)` once to turn that cubic dictionary into a path dictionary. Returned paths carry Kurvst's explicit wire path in `path`, plus Typst-friendly `points`, cubic `curves`, line `segments`, and `length` fields for drawing or inspection. Rust converts the wire path to Kurbo's `BezPath` internally.

```
#let path = kurvst.cubic-path(..segment)
#let trimmed = kurvst.trim-path(path, start-outset: 0.2, end-outset: 0.1)
#let shifted = kurvst.parallel-path(trimmed, distance: 0.15)
```

A typical chain keeps the returned dictionaries intact until the final drawing step:

```
#let base = kurvst.hobby-through(demo-start, demo-through, demo-end)
#let trimmed = kurvst.trim-path(base, start-outset: 0.2, end-outset: 0.1)
#let shifted = kurvst.parallel-path(trimmed, distance: 0.15)

#cetz.canvas({
  kurvst.cetz-path(base, stroke: rgb("#bbbbbb") + 0.4pt)
  kurvst.cetz-path(trimmed, stroke: rgb("#d72638") + 0.6pt)
  kurvst.cetz-path(shifted, stroke: rgb("#1b7f4c") + 0.6pt)
})
```

Paths And Trimming

`cubic-path` constructs a path from one cubic Bezier. `trim-path` trims by curve distance from the start and/or end and returns another path dictionary.

```
#let path = kurvst.cubic-path(..segment)
#let trimmed = kurvst.trim-path(path, start-outset: 0.15, end-outset: 0.1)
```

`hobby-through(start, through, end)` builds a smooth open curve through three points; its `curves` field contains the two cubic halves split at `through`. `hobby-spline(points)` does the same for any open point sequence with at least two points. Both return path dictionaries and can be passed directly to path-level helpers.

```
#let spline = kurvst.hobby-spline((
  (x: 0.0, y: 0.0),
  (x: 0.9, y: 0.8),
  (x: 1.8, y: -0.3),
  (x: 2.8, y: 0.4),
))
#let shifted = kurvst.parallel-path(spline, distance: 0.14)
#let wiggle = kurvst.pattern-path(
  spline,
  pattern: kurvst.wave(samples-per-period: 24),
```

```
  amplitude: 0.1,
  wavelength: 0.55,
)
#cetz.canvas({
  kurvst.cetz-path(spline, stroke: rgb("#bbbbbb") + 0.45pt)
  kurvst.cetz-path(shifted, stroke: rgb("#1b7f4c") + 0.6pt)
  kurvst.cetz-pattern(wiggle, stroke: rgb("#355c9a") + 0.55pt)
})
```

`edge-halves` combines the three-point Hobby helper with endpoint trimming for graph-edge style geometry.

Path Patterns

`pattern-path` generates a one-dimensional pattern along any path dictionary. Built-ins are regular Typst dictionaries:

```
#let wave = kurvst.wave()
#let zigzag = kurvst.zigzag()
#let coil = kurvst.coil(longitudinal-scale: 1.6)
```

String names `"wave"`, `"zigzag"`, and `"coil"` are accepted for convenience, but they are resolved in Typst before calling wasm.

Custom patterns use the same object shape:

```
#let hook = (
  kind: "points",
  name: "hook",
  interpolation: "linear",
  points: (
    (at: 0, x: 0, y: 0),
    (at: 0.25, x: 0.15, y: 1),
    (at: 0.5, x: 0, y: 0),
    (at: 0.75, x: -0.15, y: -1),
    (at: 1, x: 0, y: 0),
  ),
)
```

`at` is the phase position inside one wavelength, from `0` to `1`. The `x` coordinate offsets along the path tangent and `y` offsets along the path normal; both are scaled by `amplitude`. If `at` is omitted, points are spaced evenly. Use `interpolation: "linear"` for corners and `"smooth"` for a spline through sampled points.

`endpoint-ramp: true` tapers amplitude at anchored endpoints. This is useful for coils, whose longitudinal offset would otherwise put the first and last visible points inside the turn near nodes.

```
#let base = kurvst.cubic-path(..segment)
#let path = kurvst.pattern-path(
  base,
  pattern: hook,
  amplitude: 0.15,
  wavelength: 0.7,
)
#native-scene({
  native-cubics(base.curves, stroke: rgb("#c8c8c8") + 0.45pt)
  native-polyline(path.points, stroke: rgb("#1b7f4c") + 0.8pt)
})
```

Parallel Paths

`parallel-path` uses Kurbo's offset curve fitter to produce a path at a fixed normal distance from the source path. Positive distances follow the left normal of the path direction; negative distances follow the right normal. The output has the same drawable shape as pattern paths: `points`, fitted cubic `curves`, straight `segments`, and `length`. `start-outset` and `end-outset` trim the fitted path by arc length after the offset is computed.

```
#let base = kurvst.cubic-path(..segment)
#let left = kurvst.parallel-path(base, distance: 0.18, start-outset: 0.35, end-outset: 0.35)
#let right = kurvst.parallel-path(base, distance: -0.18)
```

```
#native-scene({
  native-cubics(base.curves, stroke: rgb("#c8c8c8") + 0.45pt)
  native-cubics(left.curves, stroke: rgb("#1b7f4c") + 0.8pt)
  native-cubics(right.curves, stroke: rgb("#355c9a") + 0.8pt)
})
```

Path Layers

`layer-path` is a convenience wrapper for drawing derived visible paths. It combines side-aware offsetting, endpoint outsets, and centered shortening into one path-in/path-out operation. `length` is a fixed visible arc length, `ratio` is a fraction of the input path length, and `resolve-length` decides how to combine them. The default `"min"` keeps whichever limit is shorter.

```
#let base = kurvst.hobby-spline((
  (x: 0.0, y: 0.0),
  (x: 0.9, y: 0.8),
  (x: 1.8, y: -0.3),
  (x: 2.8, y: 0.4),
))
#let layer = kurvst.layer-path(
  base,
  offset: 0.16,
  length: 1.6,
  ratio: 0.5,
  resolve-length: "min",
)
#cetz.canvas({
  kurvst.cetz-path(base, stroke: rgb("#bbbbbb") + 0.45pt)
  kurvst.cetz-path(layer, stroke: rgb("#1b7f4c") + 0.8pt)
})
```

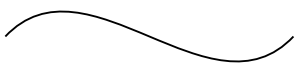
Use `side-point` to choose the sign of the offset from a point on the desired side of the path. Drawing packages can use this to place derived layers on the same side as an edge label without knowing anything about the physics or graph style that requested the layer.

Native Drawing Primitives

Kurvst returns dictionaries and arrays. The core geometry can be drawn without CeTZ by mapping points to Typst lengths and feeding them to native `curve` components:

```
#let point(p) = (p.x * 36pt, p.y * 36pt)
#let draw-path(path) = curve(
  stroke: black + 0.7pt,
  ..path.curves.map(segment => (
    curve.move(point(segment.start)),
    curve.cubic(point(segment.ctrl-a),
point(segment.ctrl-b), point(segment.end)),
  )).flatten(),
)

#draw-path(kurvst.cubic-path(..segment))
```



The generated reference below keeps drawing code inline. It only uses the small common helpers above for coordinate scaling, native `curve` construction, point markers, and fixed-size preview blocks.

CeTZ Interoperability

The CeTZ helpers are thin adapters over the same returned geometry. Use them when the surrounding document already lives in a CeTZ canvas, or when you want CeTZ path merging and styling:

```
#cetz.canvas({
  let base = kurvst.cubic-path(..segment)
  kurvst.cetz-path(base, stroke: rgb("#d72638") +
0.6pt)

  let path = kurvst.pattern-path(
```

```
    base,
    pattern: kurvst.coil(longitudinal-scale: 1.6),
    amplitude: 0.12,
    wavelength: 0.55,
  )
  kurvst.cetz-pattern(path, stroke: rgb("#355c9a")
+ 0.55pt)
})
```

Generated Reference

kurvst

- _resolve-length-target()
- center-outset()
- cetz-edge-halves()
- cetz-path()
- cetz-pattern()
- coil()
- cubic-path()
- cubic-point()
- cubic-tangent()
- edge-halves()
- hobby-spline()
- hobby-through()
- layer-path()
- line-segment()
- parallel-path()
- path-length()
- path-segments()
- pattern-path()
- trim-path()
- wave()
- zigzag()

Variables

- layer-defaults

_resolve-length-target

Resolve a fixed and relative visible path length.

`length` is interpreted as a fixed arc length. `ratio` is interpreted as a fraction of `base-length`. `method` may be `"min"`/`"shorter"`, `"max"`/`"longer"`, `"length"`/`"fixed"`, `"ratio"`/`"relative"`, `"none"`/`"full"`, or a function receiving `(base-length, length, ratio)`.

Parameters

```
_resolve-length-target(
  base-length,
  length,
  ratio,
  method
) -> none int float
```

center-outset

Compute the symmetric trim needed to center a shorter path layer.

Parameters

```
center-outset(
  base-length,
  length,
  ratio,
  resolve-length,
  start-outset,
  end-outset
) -> int float
```

cetz-edge-halves

Draw the two halves of a laid-out graph edge through CeTZ.

`source-style` applies from the source node to the edge layout point, and `sink-style` applies from the edge layout point to the sink node.

```
#cetz.canvas({
  kurvst.cetz-edge-halves(
    demo-edge,
    demo-nodes,
    source-outset: 0.2,
    sink-outset: 0.2,
    source-style: (stroke: rgb("#d72638") +
0.6pt),
    sink-style: (stroke: rgb("#355c9a") +
0.6pt),
  )
})
```

Parameters

```
cetz-edge-halves(
  edge,
  nodes,
  unit,
  omega,
  source-outset,
  sink-outset,
  accuracy,
  source-style,
  sink-style
) -> content
```

cetz-path

Draw a path dictionary through CeTZ.

```
#let path = kurvst.parallel-path(kurvst.cubic-
path(..demo-segment), distance: 0.18)
#cetz.canvas({
  kurvst.cetz-path(path, stroke: rgb("#355c9a")
+ 0.55pt)
})
```

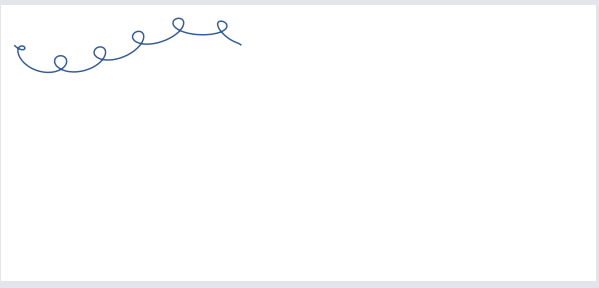
Parameters

```
cetz-path(
  path,
  unit,
  ..style
) -> content
```

cetz-pattern

Draw a sampled path pattern.

```
#let path = kurvst.pattern-path(
  kurvst.cubic-path(..demo-segment),
  pattern: kurvst.coil(longitudinal-scale: 1.6),
  amplitude: 0.12,
  wavelength: 0.55,
)
#cetz.canvas({
  kurvst.cetz-pattern(path, stroke:
rgb("#355c9a") + 0.55pt)
})
```



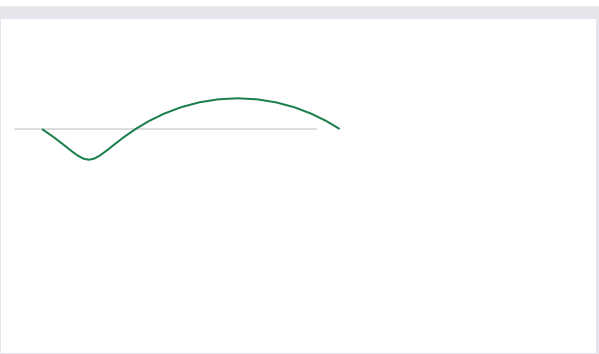
Parameters

```
cetz-pattern(
  path,
  unit,
  ..style
) -> content
```

coil

A smooth coil path pattern.

```
#let pattern = kurvst.coil(samples-per-period:
24, longitudinal-scale: 1.6)
#let points = pattern.points.map(point => (
  x: point.at * 3.1 + point.x * 0.18,
  y: 1 + point.y * 0.32,
))
#native-scene({
  native-polyline(((x: 0.0, y: 1.0), (x: 3.15,
y: 1.0)), stroke: rgb("#bbbbbb") + 0.35pt)
  native-polyline(points, stroke: rgb("#1b7f4c")
+ 0.75pt)
})
```



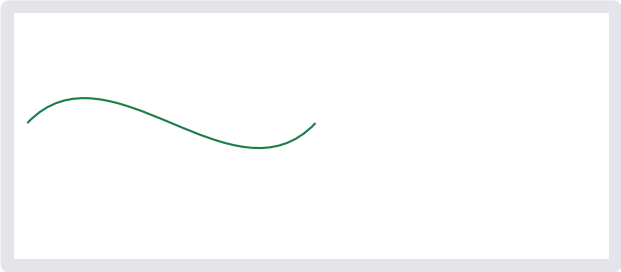
Parameters

```
coil(
  samples-per-period,
  longitudinal-scale
) -> dictionary
```

cubic-path

Build a path dictionary from one cubic Bezier.

```
#let path = kurvst.cubic-path(..demo-segment)
#native-scene({
  native-cubics(path.curves, stroke:
rgb("#1b7f4c") + 0.8pt)
})
```



Parameters

```
cubic-path(
  start,
  end,
  ctrl-a,
  ctrl-b,
  accuracy
) -> dictionary
```

cubic-point

Evaluate a cubic segment at parameter `t`.

Parameters

```
cubic-point(
  segment,
  t
) -> dictionary
```

cubic-tangent

Evaluate the tangent of a cubic segment at parameter `t`.

Parameters

```
cubic-tangent(
  segment,
  t
) -> dictionary
```

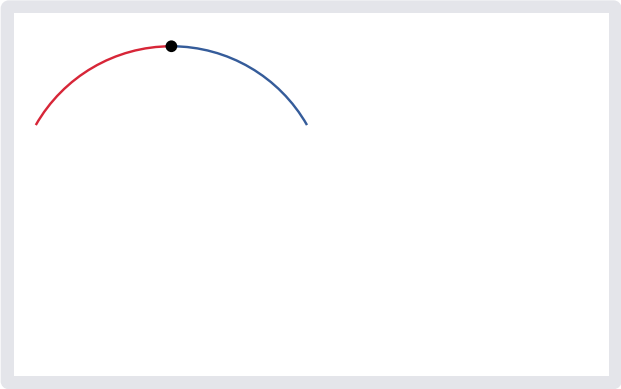
edge-halves

Split a laid-out graph edge into source and sink half-edge paths.

The returned dictionary has `source`, `sink`, and `curve`. The split point is the edge layout point, so the two half-edges join smoothly there.

```
#let halves = kurvst.edge-halves(
  demo-edge,
  demo-nodes,
  source-outset: 0.2,
  sink-outset: 0.2,
)
#native-scene({
  native-cubics(halves.source.curves, stroke:
rgb("#d72638") + 0.9pt)
  native-cubics(halves.sink.curves, stroke:
```

```
rgb("#355c9a") + 0.9pt)
  native-dot(demo-through, fill: black)
})
```



Parameters

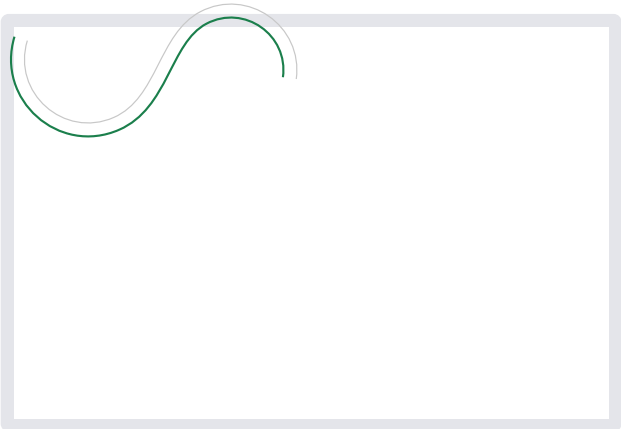
```
edge-halves(
  edge,
  nodes,
  omega,
  source-outset,
  sink-outset,
  accuracy
) -> dictionary
```

hobby-spline

Build an open Hobby spline through two or more points.

The returned dictionary is a path dictionary backed by Kurvst's explicit wire path. Rust converts that wire path to Kurbo's `BezPath` internally, so it can be passed directly to `parallel-path()`, `pattern-path()`, `trim-path()`, or `cetz-path()`.

```
#let spline = kurvst.hobby-spline((
  (x: 0.0, y: 0.0),
  (x: 0.9, y: 0.8),
  (x: 1.8, y: -0.3),
  (x: 2.8, y: 0.4),
))
#let shifted = kurvst.parallel-path(spline,
distance: 0.14)
#native-scene({
  native-cubics(spline.curves, stroke:
rgb("#c8c8c8") + 0.45pt)
  native-cubics(shifted.curves, stroke:
rgb("#1b7f4c") + 0.8pt)
})
```



Parameters

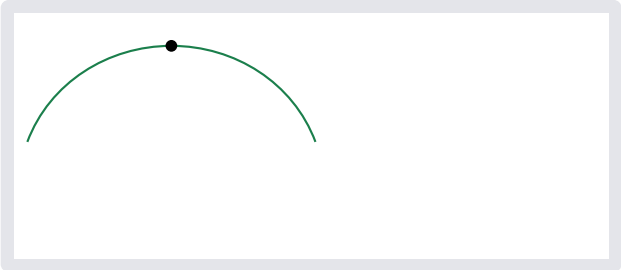
```
hobby-spline(
  points,
  omega,
  accuracy
) -> dictionary
```

hobby-through

Build an open Hobby curve through `start`, `through`, and `end`.

The returned dictionary is a path dictionary. For this three-point variant, `curves` contains the two cubic halves meeting at `through`.

```
#let path = kurvst.hobby-through(demo-start,
demo-through, demo-end, omega: 1.2)
#native-scene({
  native-cubics(path.curves, stroke:
rgb("#1b7f4c") + 0.8pt)
  native-dot(demo-through, fill: black)
})
```



Parameters

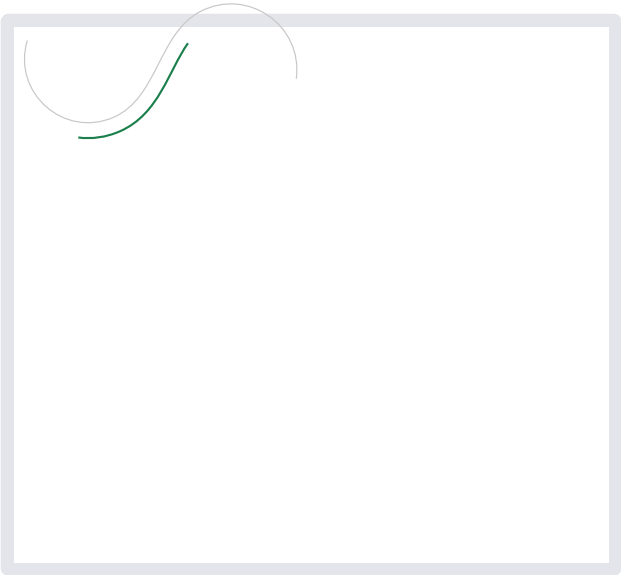
```
hobby-through(
  start,
  through,
  end,
  omega,
  accuracy
) -> dictionary
```

layer-path

Build a derived visible path layer.

`layer-path` combines the common operations needed by drawing packages: optional side-aware offsetting, endpoint trimming, and centered shortening by a fixed `length`, a relative `ratio`, or both. The return value is a normal Kurvst path dictionary that can be passed to `pattern-path()`, `parallel-path()`, `trim-path()`, or `cetz-path()`.

```
#let base = kurvst.hobby-spline((
  (x: 0.0, y: 0.0),
  (x: 0.9, y: 0.8),
  (x: 1.8, y: -0.3),
  (x: 2.8, y: 0.4),
))
#let layer = kurvst.layer-path(
  base,
  offset: 0.16,
  length: 1.6,
  ratio: 0.5,
  resolve-length: "min",
)
#native-scene({
  native-cubics(base.curves, stroke:
rgb("#c8c8c8") + 0.45pt)
  native-cubics(layer.curves, stroke:
rgb("#1b7f4c") + 0.8pt)
})
```



Parameters

```
layer-path(
  path,
  offset,
  length,
  ratio,
  resolve-length,
  start-outset,
  end-outset,
  side-point,
  accuracy,
  optimize
) -> dictionary
```

line-segment

Build a cubic segment dictionary for a straight line.

Parameters

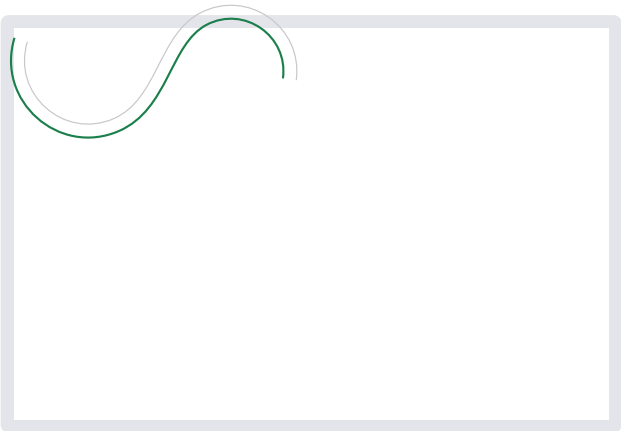
```
line-segment(
  start,
  end
) -> dictionary
```

parallel-path

Generate a parallel path for a path.

Pass any path returned by this module, such as `hobby-spline()`. The result is a drawable path dictionary with the fitted parallel path and its explicit wire path.

```
#let spline = kurvst.hobby-spline((
  (x: 0.0, y: 0.0),
  (x: 0.9, y: 0.8),
  (x: 1.8, y: -0.3),
  (x: 2.8, y: 0.4),
))
#let parallel = kurvst.parallel-path(spline,
distance: 0.14)
#native-scene({
  native-cubics(spline.curves, stroke:
rgb("#c8c8c8") + 0.45pt)
  native-cubics(parallel.curves, stroke:
rgb("#1b7f4c") + 0.8pt)
})
```



Parameters

```
parallel-path(
  path,
  distance,
  start-outset,
  end-outset,
  accuracy,
  optimize
) -> dictionary
```

path-length

Compute the arc length of a path dictionary.

If the path already carries a `length` field, that value is returned. Otherwise the length is computed from the drawable segments.

Parameters

```
path-length(
  path,
  accuracy
) -> int float
```

path-segments

Return drawable cubic segments for any Kurvst path dictionary.

Line segments and sampled point paths are converted to equivalent cubic line segments so downstream renderers can use one code path.

Parameters

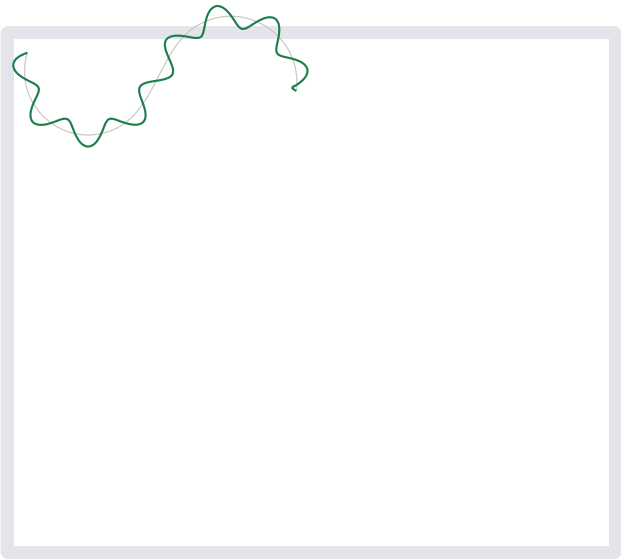
```
path-segments(path) -> array
```

pattern-path

Generate a sampled 1D pattern along a path.

`pattern` may be `wave()`, `zigzag()`, `coil()`, a compatible string name, or a point pattern: `(kind: "points", interpolation: "linear" or "smooth", points: ((at: 0, x: 0, y: 0), ...))`. The whole input path is sampled continuously, so pattern phase does not restart at cubic segment boundaries.

```
#let spline = kurvst.hobby-spline((
  (x: 0.0, y: 0.0),
  (x: 0.9, y: 0.8),
  (x: 1.8, y: -0.3),
  (x: 2.8, y: 0.4),
))
#let path = kurvst.pattern-path(
  spline,
  pattern: kurvst.wave(samples-per-period: 24),
  amplitude: 0.12,
  wavelength: 0.55,
)
#native-scene({
  native-cubics(spline.curves, stroke:
rgb("#c8c8c8") + 0.45pt)
  native-cubics(path.curves, stroke:
rgb("#1b7f4c") + 0.8pt)
})
```



Parameters

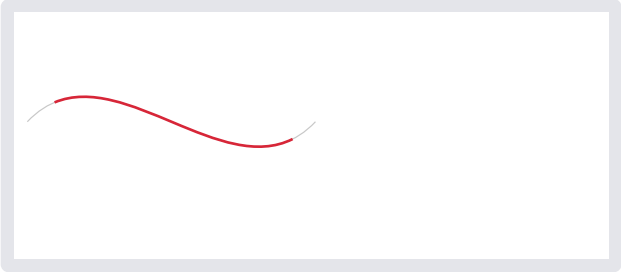
```
pattern-path(
  path,
  pattern,
  amplitude,
  wavelength,
  phase,
  samples-per-period,
  coil-longitudinal-scale,
  anchor-start,
  anchor-end,
  accuracy
) -> dictionary
```

trim-path

Trim a path by curve distance from either end.

The returned dictionary is another path dictionary with `path`, `points`, `curves`, `segments`, and `length`.

```
#let base = kurvst.cubic-path(..demo-segment)
#let trimmed = kurvst.trim-path(base, start-
outset: 0.35, end-outset: 0.3)
#native-scene({
  native-cubics(base.curves, stroke:
rgb("#c8c8c8") + 0.45pt)
  native-cubics(trimmed.curves, stroke:
rgb("#d72638") + 1pt)
})
```



Parameters

```
trim-path(
  path,
  start-outset,
  end-outset,
  accuracy
) -> dictionary
```

wave

A smooth sinusoidal path pattern.

```
#let pattern = kurvst.wave(samples-per-period:
24)
#let points = pattern.points.map(point => (
  x: point.at * 3.1 + point.x * 0.18,
  y: 1 + point.y * 0.32,
))
#native-scene({
  native-polyline(((x: 0.0, y: 1.0), (x: 3.15,
y: 1.0)), stroke: rgb("#bbbbbb") + 0.35pt)
  native-polyline(points, stroke: rgb("#1b7f4c")
+ 0.75pt)
})
```

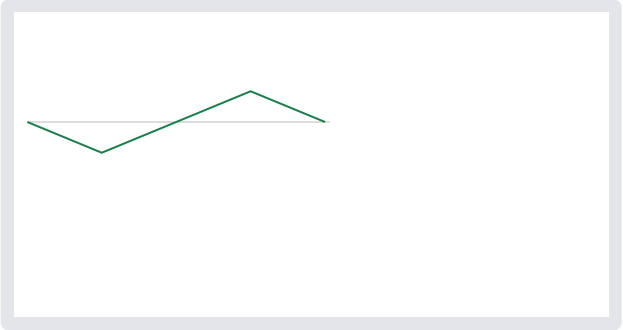
Parameters

```
wave(samples-per-period) -> dictionary
```

zigzag

A straight-segment triangular path pattern.

```
#let pattern = kurvst.zigzag()
#let points = pattern.points.map(point => (
  x: point.at * 3.1 + point.x * 0.18,
  y: 1 + point.y * 0.32,
))
#native-scene({
  native-polyline(((x: 0.0, y: 1.0), (x: 3.15,
y: 1.0)), stroke: rgb("#bbbbbb") + 0.35pt)
  native-polyline(points, stroke: rgb("#1b7f4c")
+ 0.75pt)
})
```



Parameters

`zigzag() -> dictionary`

layer-defaults `dictionary`

Default geometry options for derived path layers.

These defaults are shared by `layer-path()` and drawing packages that build on Kurvst. The dictionary shape intentionally mirrors the public function arguments so callers can merge user styles into it and unpack the result.